//Program on Multilevel Inheritance :

```
package com.rest.multi_level_inheritance;

class Student
{
    protected int id;
    protected String name;
    protected String address;

    public StudentInit(int id, String name, String address)
    {
        super();
        this.id = id;
        this.name = name;
        this.address = address;
    }

    @override
    public String toString();
    {
        return "Student [id= " + id + ", name= " + name + ", address= " + address + "]";
    }
}

class Science extends Student
{
    protected int physics;
    protected int chemistry;

    public ScienceInit(int id, String name, String address, int physics, int chemistry)
    {
        super(id, name, address);
        this.physics = physics;
        this.chemistry = chemistry;
    }

    @override
    public String toString();
    {
        return super.toString()+"science [physics= " + physics + ", chemistry= " + chemistry + "]";
    }
}

class PCM extends Science
{
    protected int math;

    public PCMInit(int id, String name, String address, int physics, int chemistry, int math)
    {
        super(id, name, address, physics, chemistry);
        this.math = math;
    }

    public void getStudyMarks()
    {
        int[ total = this.physics + this.chemistry + this.math;
        System.out.println("Total Marks is :" +total);
    }

    @override
    public String toString();
    {
        return super.toString()+ "PCM [math= " + math + "]";
    }
}

public class MultiLevelDemo
{
    public static void main(String[] args)
    {
        PCM s = new PCMInit(1, "Sunil", "Ramnagar", 90, 80, 98);
        System.out.println(s);
        s.getStudyMarks();
    }
}

Assignment :
//Program on Hybrid Inheritance :
```

HOW MANY WAYS WE CAN INITIALIZE THE OBJECT PROPERTIES ?

The following are the ways to initialize the object properties

```
public class Test
{
    int x,y;
}
```

1) At the time of declaration :

Example :

```
public class Test
{
    int x = 10;
    int y = 20;
}

Test t1 = new Test();   [x = 10  y = 20]
Test t2 = new Test();   [x = 10  y = 20]
```

Here the drawback is all objects will be initialized with same value.

2) By using Object Reference :

```
public class Test
{
    int x,y;
}

Test t1 = new Test();   [x = 10;   t1.x=20;
Test t2 = new Test();   [x = 20;   t2.x=40;
```

Here we are getting different values with respect to object but here the program becomes more complex.

3) By using methods :

A) First Approach (Method without Parameter) :

```
public class Test
{
    int x,y;

    public void setData()
    {
        x = 100;  y = 200;
    }
}

Test t1 = new Test();  t1.setData();  [x = 100   y = 200]
Test t2 = new Test();  t2.setData();  [x = 100   y = 200]
```

Here also, all the objects will be initialized with same value.

B) Second Approach (Method with Parameter)

```
public class Test
{
    int x,y;

    public void setData(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}

Test t1 = new Test();  t1.setData(12,78);  [x = 12  y = 78]
Test t2 = new Test();  t2.setData(15,25);  [x = 15  y = 25]
```

Here the Drawback is initialization and re-initialization both are done in two steps. So, Constructor introduced.

4) By using Constructor

A) First Approach (No Argument Constructor)

```
public class Test
{
    int x,y;

    public Test();  //All the objects will be initialized with
                    same value
    {
        x = 0;  y = 0;
    }
}

Test t1 = new Test();   [x = 0   y = 0]
Test t2 = new Test();   [x = 0   y = 0]
```

B) Second Approach (Parameterized Constructor)

```
public class Test
{
    int x,y;

    public Test(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}

Test t1 = new Test(12,78);   [x = 12   y = 78]
Test t2 = new Test(15,25);   [x = 15   y = 25]
```

This is the best way to initialize our instance variable because variable initialization and variable re-initialization both will be done in the same line as well as all the objects will be initialized with different values.

C) Third Approach (Copy Constructor)

```
    public class Manager
    {
        private int managerId;
        private String managerName;

        public Manager(Employee emp)
        {
            this.managerId = emp.getEmployeeId();
            this.managerName = emp.getEmployeeName();
        }
    }
```

Here with the help of Object reference (Employee class) we are initializing the properties of Manager class. *(Copy Constructor)*

D) By using instance block (Instance Initializer)

```
public class Test
{
    int x,y;

    public Test();

    {
        System.out.println(); //100
        System.out.println(); //200
    }

    //Instance block

    {
        x = 100;
        y = 200;
    }
}

Test t1 = new Test();
Test t2 = new Test();
```

5) By using super keyword :

```
class Super
{
    int x,y;

    public Super(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}

class Sub extends Super
{
    Sub();

    {
        super(100,200); //Initializing the properties of super class
    }

    new Sub();
}
```

*** Why Java does not support Multiple Inheritance :

Multiple Inheritance is a situation where a sub class wants to inherit the properties two or more than two super classes.

In every constructor we have super() or this(). When compiler will add super() to the first line of the constructor then we have an ambiguity issue that super() will call which super class constructor as both are in single line.

It is also known as Diamond Problem in java so the final conclusion is we can't achieve multiple inheritance using classes but same we can achieve by using interface (interface does not contain any constructor)

JVM Architecture with class Loader sub system :

* The entire JVM Architecture is divided into 3 sections

1) Class Loader subsystem
2) Runtime Data Areas (Memory Areas)
3) Execution Engine

Class Loader subsystem :

* Internally Class Loader subsystem performs the following operation :
a) LOADING b) LINKING c) INITIALIZATION